

Quick-Start Implementation Guide

PV-Battery Microgrid EMS — Phases 1-4, Step-by-Step MATLAB Build Order

Abhi — B.Tech Electrical Engineering, Assam Engineering College

September 2026

Contents

1 How to Use This Document	1
2 Module 0 — Setup (half a day)	2
3 Module 1 — Phase 1: Rule-Based EMS	3
4 Module 2 — Phase 2: Naive Cost-Minimizing LP	4
5 Module 3 — Quantify the Damage (Phase 2 Post-Hoc)	5
6 Module 4 — Phase 3: Degradation-Aware Optimization	6
7 Module 5 — Phase 4: Minimal Benchmarking Table	7
8 What’s Deliberately Left Out (For Now)	8

1 How to Use This Document

This is a **short, buildable companion** to the full *Technical Implementation Blueprint*. It covers only **Phases 1-4** (baseline → naive LP → degradation-aware LP/QP → benchmarking), because those are what you should build first and validate before touching MPC, Simulink, or the dashboard. Each module below is self-contained: goal, build steps, the one equation you actually need, a minimal MATLAB skeleton, and a “done when” checklist. Where you want the full derivation, parameter tables, or complete commented code, a **[Blueprint Part N]** reference points to the matching section of the big document — don’t re-derive things here that are already there.

Do not start the next module until the current one’s checklist passes. This is the actual dependency chain, not a suggestion.

2 Module 0 — Setup (half a day)

Goal: one params struct and two synthetic data generators, so every later module has something real to run against.

Steps:

1. Create folders: /models /ems /optimization /degradation /analysis /plots /main — skip /mpc /dashboard for now. [Blueprint Part 17]
2. Write system_params.m — a single function returning a struct with E_B, eta_ch, eta_dis, dT, SOC0, SOC_min, SOC_max, P_ch_max, P_dis_max, P_grid_imp_max, P_grid_exp_max. Fill in **your actual numbers**, not placeholders. [Blueprint Part 2.3 for the parameter table]
3. Write generate_pv_profile.m and generate_load_profile.m (synthetic, parametric — full code in [Blueprint Part 18.3]). Plot them. If they don't look like a PV/load day, stop and fix before moving on.

Done when: params = system_params(); pv = generate_pv_profile(96,0.25,params.Ppv Rated, plot(pv) runs and looks right.

3 Module 1 — Phase 1: Rule-Based EMS

Goal: a dumb, honest baseline. PV feeds load first, surplus charges battery then exports; deficit is met by battery then grid.

One equation you need (SOC update):

$$SOC(k+1) = SOC(k) + \frac{\Delta T}{E_B} \left(\eta_{ch} P_{ch}(k) - \frac{P_{dis}(k)}{\eta_{dis}} \right)$$

Build steps:

1. Write `rule_based_ems.m` implementing the if/else logic below (full commented version: [Blueprint Part 5]).
2. Run it on your Module 0 profiles.
3. Write a **10-line** validation check inline — don't build the full validation suite yet, just confirm power balance residual ≈ 0 and SOC stays in bounds.

Minimal skeleton:

```
function [P_ch,P_dis,P_gimp,P_gexp,SOC] = rule_based_ems(P_pv,P_load,p)
N = length(P_pv); SOC = zeros(N+1,1); SOC(1) = p.SOC0;
P_ch=zeros(N,1); P_dis=zeros(N,1); P_gimp=zeros(N,1); P_gexp=zeros(N,1);
for k = 1:N
    net = P_pv(k) - P_load(k);
    if net >= 0
        P_ch(k) = min([net, p.P_ch_max, (p.SOC_max-SOC(k))*p.E_B/(p.eta_ch*p.dT)]);
        P_gexp(k) = min(net - P_ch(k), p.P_grid_exp_max);
    else
        P_dis(k) = min([-net, p.P_dis_max,
            ↪ (SOC(k)-p.SOC_min)*p.E_B*p.eta_dis/p.dT]);
        P_gimp(k) = min(-net - P_dis(k), p.P_grid_imp_max);
    end
    SOC(k+1) = SOC(k) + p.dT/p.E_B*(p.eta_ch*P_ch(k) - P_dis(k)/p.eta_dis);
end
end
```

Done when: dispatch plot shows battery charging on surplus / discharging on deficit, $\max(\text{abs}(\text{balance residual})) < 1\text{e-}6$, and SOC never leaves $[SOC_{\min}, SOC_{\max}]$.

4 Module 2 — Phase 2: Naive Cost-Minimizing LP

Goal: replace the if/else with a linprog call that minimizes grid cost under a ToD tariff — no degradation awareness yet, on purpose.

One equation you need (objective):

$$\min \sum_k [C_{import}(k)P_{grid,imp}(k) - C_{export}(k)P_{grid,exp}(k)]\Delta T$$

Build steps:

1. Write `calculate_tariff.m` — start with 'flat' mode, get the LP working, *then* switch to 'tod' mode. [Blueprint Part 7]
2. Write `build_lp_problem.m` — decision vector `[P_gimp; P_gexp; P_ch; P_dis]`, equality constraint = power balance, inequality constraints = SOC bounds via cumulative sums. This is the fiddliest part of the whole project — go slow. [Blueprint Part 6, full commented linprog boilerplate]
3. Write `run_lp_ems.m` (thin wrapper around `linprog`, check `exitflag == 1`).
4. Re-run your Module 1 validation check on the LP output too.

Skeleton (index bookkeeping only — get the real constraint matrices from Blueprint Part 6):

```
function lp = build_lp_problem(P_pv, P_load, tariff, p)
N = length(P_pv);
idx.imp=1:N; idx.exp=N+1:2*N; idx.ch=2*N+1:3*N; idx.dis=3*N+1:4*N;
f = zeros(4*N,1);
f(idx.imp) = tariff.C_import(:)*p.dT;
f(idx.exp) = -tariff.C_export(:)*p.dT;
% Aeq/beq = power balance (Blueprint Part 6.3)
% A/b     = SOC bounds via tril(ones(N)) cumulative sum (Blueprint Part 6.4)
% lb/ub   = zeros / rated power limits (Blueprint Part 6.5)
lp = struct('f',f,'idx',idx,'N',N); % + Aeq,beq,A,b,lb,ub
end
```

Done when: `linprog` converges (`exitflag==1`), total grid cost is **lower** than Module 1's, and the same validation check still passes.

5 Module 3 — Quantify the Damage (Phase 2 Post-Hoc)

Goal: prove, with numbers, that Module 2's naive LP cycles the battery harder than Module 1 — this is the finding that justifies Module 4.

One equation you need:

$$C_{deg} = Cost_{replacement} \times \frac{\text{throughput (kWh)}}{2 E_B LifeCycles(DoD)}$$

Build steps:

1. Write `calculate_degradation.m` — throughput-based, using a simple power-law $LifeCycles(DoD) = A * DoD^{-b}$ (get literature values for your cell chemistry). [Blueprint Part 11]
2. Run it on both Module 1 and Module 2's `P_ch`, `P_dis` outputs.
3. Tabulate: grid cost and degradation cost, side by side, for both.

Done when: you can show Module 2 has lower grid cost **and** higher degradation cost than Module 1. If degradation cost is *not* higher, something's off — recheck before moving on (see [Blueprint Part 21] expected-behaviour table).

6 Module 4 — Phase 3: Degradation-Aware Optimization

Goal: one combined objective, solved with quadprog, that balances the two costs from Module 3.

One equation you need:

$$\min \sum_k \left[C_{import}(k) P_{grid,imp}(k) \Delta T - C_{export}(k) P_{grid,exp}(k) \Delta T + \alpha \beta (P_{ch}(k) + P_{dis}(k))^2 \right]$$

Build steps:

1. Reuse Module 2's `build_lp_problem.m` for `Aeq/A/b/lb/ub` — only the objective changes.
2. Add the quadratic penalty matrix `H` (full code: [Blueprint Part 12.4]) and call `quadprog(H, f, A, b, Aeq, beq, lb, ub)` instead of `linprog`.
3. Sweep α over a handful of values (e.g. 0, 0.1, 1, 10) and re-run Module 3's degradation calc on each result.
4. Plot grid cost vs. degradation cost across the sweep — this curve **is** your Phase 3 deliverable.

Done when: the trade-off curve is roughly monotonic (higher $\alpha \rightarrow$ lower degradation cost, higher grid cost) and at least one α gives a total cost (grid + degradation) lower than Module 2's naive total.

7 Module 5 — Phase 4: Minimal Benchmarking Table

Goal: one table, three controllers, one scenario — don't build the full scenario/sensitivity matrix yet.

Steps:

1. Run Modules 1, 2, and 4 (pick one good α) on the **same** PV/load/tariff input.
2. Tabulate: grid cost, degradation cost, total cost, self-sufficiency (`calculate_metrics.m`, [Blueprint Part 10]) for all three.
3. Confirm the ordering matches expectation: naive LP = lowest grid cost / highest degradation; degradation-aware = lowest **total** cost. [Blueprint Part 21 has the full expected-behaviour table if this doesn't hold.]

Done when: you have one clean comparison table you could paste directly into a report. At this point you have a complete, validated, defensible mini-project even before MPC/Simulink/dashboard exist — everything past here (full scenario matrix, sensitivity sweeps, MPC, forecast error, Simulink, dashboard) is in the full blueprint whenever you're ready to extend.

8 What's Deliberately Left Out (For Now)

- **Peak-shaving extension** — [Blueprint Part 9]
- **Full scenario matrix / sensitivity sweeps** — [Blueprint Part 13-14]
- **MPC / rolling-horizon control** — [Blueprint Part 15-16]
- **Simulink architecture** — [Blueprint Part 19]
- **Full validation suite, App Designer dashboard, report boilerplate, figure plan, viva prep, novelty assessment** — [Blueprint Parts 20, 22-26]

Come back to this list once Module 5 passes — the full blueprint has all of it ready to go.